

Diagnostic Assist

Technical assignment submission | Riddhi Shah

Code: github.com/Riddhikshah21/diagnostic-assist

This submission describes the intended product and the production design. The implemented Python component is multilingual evidence retrieval.

1. What the product actually is

When I first read the assignment, I pictured an assistant that searches old service cases and helps someone understand what might be wrong with a machine. I would make the service dispatcher the main user for the first version. The customer describes the problem over the phone; the dispatcher uses the assistant; the technician receives the handoff and confirms the cause onsite.

How the conversation works

The dispatcher selects the equipment and enters the customer's description in English, German, French, Italian or mixed language. The interface shows what it understood so the dispatcher can correct it, then shows similar historical cases. The first useful evidence is targeted within three seconds at p95. Possible causes and a short follow-up can arrive afterward.

The dispatcher records the customer's answer or selects Unknown. A useful answer updates the conversation state and retrieval query. Unknown adds no diagnostic evidence. The handoff contains symptoms, collected answers, possible causes, supporting and conflicting cases, suggested technician checks and parts used in similar repairs. The intended product includes cause probabilities; the current CLI returns historical cases and ranking scores only.

The conversation ends when the dispatcher has enough information to arrange the visit, the next check requires a technician, the customer cannot answer, or the evidence cannot narrow the issue further. I would not stop solely because a cause passes a probability threshold.

Walkthrough: C-48211

I treat C-48211 as a new case and exclude it from historical retrieval. The dispatcher selects CX-450 and enters: "Unit won't start at all. No lights on the control panel." The intended interface confirms the no-start and dead-panel symptoms. Three relevant historical cases offer different explanations:

Possible cause	Supporting case and outcome
Main contactor failure	C-48377 (German): no start, blank display; contactor replaced.
Loose or corroded ground	C-48590: dead panel; ground strap repaired; contactor tested OK.
Failed battery	C-49040: would not power up; battery failed a load test.

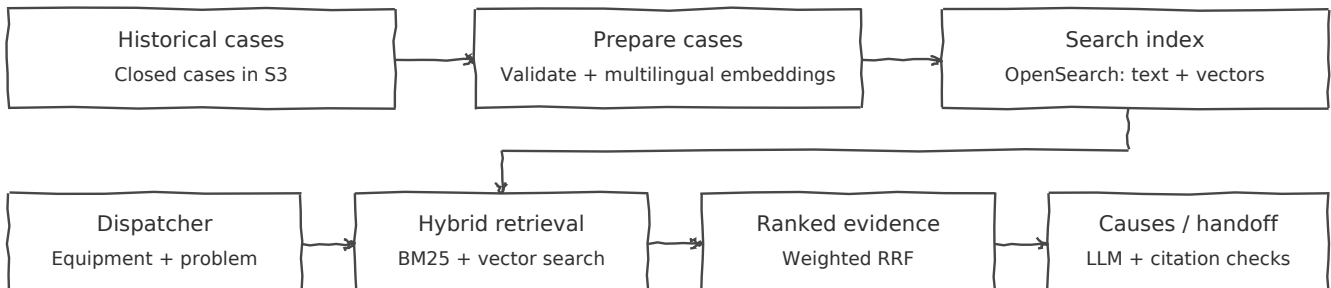
A possible follow-up is: "Has the battery been flat or needed charging recently?" A no answer is recorded but does not rule out battery failure or justify a numerical probability update on its own. Unknown also leaves that possibility open. Electrical measurements remain for the technician. The handoff lists the three possibilities and checks; CONTACTOR-M1 is a part to consider, subject to equipment compatibility. In the actual C-48211 repair, the technician found an open contactor coil. This walkthrough is proposed product behaviour, not a chatbot implemented in the CLI.

2. Technical design | 1 of 3

Pipeline and architecture

I would start with a modular application. A batch ingestion job prepares closed cases; a warm API serves retrieval and conversation requests. At this volume, separate microservices and continuous streaming would add more operational work than value.

Proposed system - rough architecture sketch



First useful evidence: target p95 < 3s. Causes and follow-up arrive afterward.

What is stored

Versioned S3 objects preserve the original records. OpenSearch stores customer descriptions, multilingual vectors and equipment metadata for retrieval, with outcome fields available for evidence assembly. The original cases remain unchanged. A production search record would look like this:

```
{
  "case_id": "C-48590",
  "equipment_family": "Air Compressor CX",
  "equipment_type": "CX-450",
  "created_at": "2026-01-27T09:41:00Z",
  "language": "en",
  "customer_description": "no crank, dead panel",
  "customer_description_vector": [],
  "technician_notes": "Ground strap was loose...",
  "resolution_text": "Loose ground strap...",
  "parts_replaced": [],
  "pipeline_version": "1",
  "embedding_model_version": "configured model revision"
}
```

Only customer_description is embedded and searched in this prototype. Technician notes, resolution text and replaced parts describe later observations and repairs; I return them after retrieval as evidence. This keeps matching based on the information available at the start of a case. An outcome is not automatically ground truth: a replaced part may be incidental, and "no fault found" is not a confirmed cause.

Algorithms and matching against 100,000 cases

BM25 handles exact codes and technical terms. The tokenizer preserves codes such as E-207 and adds the compact form E207. Multilingual embeddings find similar descriptions across wording and languages. The prototype uses paraphrase-multilingual-MiniLM-L12-v2 and exact cosine similarity in a NumPy matrix. Production would use a multilingual Bedrock embedding provider and HNSW vectors in OpenSearch, with equipment metadata filters. HNSW trades some recall and memory for faster approximate search; I would compare it with exact retrieval before choosing its index parameters.

2. Technical design | 2 of 3

Fusion, scope and relevance

The prototype searches the same equipment type, takes candidates from BM25 and semantic retrieval, and combines their ranks using weighted Reciprocal Rank Fusion (RRF): $\text{score}(\text{case}) = \text{sum}(\text{weight} / (60 + \text{rank}))$. Rank fusion avoids comparing incompatible raw BM25 and cosine scales. The constant 60 is a conventional starting value; I did not tune it on this small sample.

With the default result limit of five, each retriever contributes five candidates. BM25 weight is 1.0 and semantic weight is 2.0. Candidate depth currently follows the requested result limit. I tested these choices on four manually reviewed sample queries. They are exploratory settings, not production-tuned parameters. Semantic weighting recovered a German case for a paraphrased no-start query, but overheating cases still appeared. Agreement between two retrievers does not prove relevance.

The implemented family fallback fills empty result slots from other equipment types in the same family and marks those hits. It is triggered by result count. Production would instead rerank a larger candidate set using a multilingual relevance model, expand to family evidence when type-specific relevance is weak, and return insufficient evidence if neither scope helps. The threshold must be chosen on reviewed queries, including negative examples. Reranking adds model cost and latency, so it needs a measured benefit.

Unlabelled outcomes and probabilities

A constrained LLM could propose temporary cause groups from the retrieved notes and resolutions, with case IDs and exact source spans. The backend would validate citations. Citation validity alone does not prove the cause is correct: negation, contradictory repairs and administrative closures need separate checks. I would show supporting evidence and case counts until probability estimates can be evaluated.

For production, I would create a separate, versioned cause dataset without changing the source cases. An LLM may propose labels, but domain reviewers must confirm a representative subset and mark ambiguous or unresolved outcomes. I would begin with a regularized classifier using pre-visit multilingual embeddings and equipment metadata, compare it with weighted nearest-neighbour prediction, and calibrate on held-out cases. A temporal test set would measure Brier score, log loss and reliability by language and equipment volume. Sparse types may need family-level pooling or abstention. Normalized retrieval weights are not diagnostic probabilities. This stage is not implemented.

Follow-up questions

DynamoDB would hold equipment, reported symptoms, codes, previous questions, answers and unknowns. The LLM would receive this state and relevant evidence, then return possible causes, citations and at most one follow-up question as validated JSON. I would choose questions that distinguish possibilities and avoid repeating answered or unknown questions. Customer checks must be safe and practical. A negative answer should change a cause estimate only when there is a justified relationship between that answer and the cause.

Latency target

Historical embeddings are computed offline. A warm ECS API would run BM25 and vector search concurrently, assemble a small evidence bundle and show it before slower generation. I would trace query embedding, retrieval, fusion, evidence assembly and generation separately and measure end-to-end p95 under representative concurrency. The CLI loads the model and rebuilds its in-memory index on each invocation; it does not demonstrate the three-second production target.

2. Technical design | 3 of 3

Initial load and 500 daily cases

An ECS Fargate batch job would read the initial 100,000 cases from S3 in chunks, validate with Pydantic, embed descriptions in batches and bulk-index into OpenSearch. It would checkpoint progress, retry transient failures with backoff and quarantine repeatedly failing records. The prototype loader instead fails the whole file on invalid records or duplicate IDs.

EventBridge Scheduler would start the daily incremental job. A persisted source watermark and case content hash would identify new or changed closed cases; the job advances its watermark only after successful writes. Within a versioned index, `case_id` is the document ID, so retries upsert rather than duplicate. If the model or schema changes, I would build a new index, evaluate it and switch an alias. Counts and sampled record checks would catch missing updates.

Stack and trade-offs

Area	Choice and cost
Storage and search	S3 for source and rejected records; OpenSearch Service for BM25, HNSW and filters. OpenSearch has a fixed operating cost; PostgreSQL + pgvector is an alternative if search stays simpler.
Models	Amazon Bedrock for multilingual embeddings and generation, behind provider interfaces. Less hosting work, but ongoing inference cost and provider dependence. A model change requires re-embedding and evaluation.
Backend and compute	Python, FastAPI, Pydantic, boto3 and opensearch-py on ECS Fargate. Warm capacity helps latency but costs money while idle.
State and frontend	DynamoDB for conversation state with expiry; React and TypeScript for evidence cards, follow-up answers and the handoff.
Operations	EventBridge for batch scheduling; CloudWatch and OpenTelemetry for logs and traces; Terraform for repeatable deployment.

Where the pipeline can fail

Failure	How I would find out or control it
Invalid, missing or stale records	Validation/quarantine counts, source-to-index reconciliation, watermark and index-lag monitoring.
Cross-language or code misses	Recall and ranking metrics by query language; mixed-language and code-format tests; comparison with semantic-only and BM25-only baselines.
Irrelevant evidence / rare equipment	Reviewed precision and nDCG, negative queries, metrics by equipment volume, explicit family-fallback provenance.
Negation or unsupported cause	Keep original text, validate citations and schema, review contradiction examples such as "contactor tested OK". No-fault and administrative cases cannot establish a cause.
Slow or expensive requests	Stage-level p50/p95, timeout/failure rate, token usage and cost per conversation. Return available evidence if generation fails.
Sensitive text or prompt injection	Restrict access, avoid raw case text in routine logs, and treat retrieved text as data rather than instructions.

The main cost of this design is that retrieval alone cannot resolve unlabelled or incorrect outcomes. It creates a useful evidence base, but a reviewed probability dataset and safe follow-up logic are still needed. The larger business outcome is improved first-visit repair rate, not merely higher retrieval recall.

3-5. Implementation, evaluation and cuts

3. Why I selected this component

I implemented multilingual hybrid retrieval and evidence assembly because every downstream cause, probability estimate and follow-up depends on useful historical evidence. A multilingual model alone was not enough: semantic retrieval found relevant German cases, but the original rank fusion pushed them below weaker English cases. I could inspect this failure on the sample, change fusion settings and test the result. Missing root-cause labels remain the largest data risk for probabilities; this component addresses the retrieval risk rather than claiming to solve both.

The Python package separates models, loading, retrieval and the CLI. It implements validation, BM25, real multilingual embeddings, exact cosine retrieval, weighted RRF, equipment filtering, visible family fallback and case exclusion for evaluation. Deterministic embedder replacements test retrieval orchestration; the evaluation scripts exercise the real multilingual model. Generation, reviewed cause extraction, probability calibration and relevance reranking are not implemented. Their intended inputs, outputs and next steps are described above rather than presented as working features.

4. How I would know it works

I manually selected relevant historical cases for four queries. Each historical query case is excluded from candidates, so it cannot retrieve itself. This is a small leave-one-out retrieval experiment, not exhaustive cross-validation. Relevance judgements are about symptom usefulness, not confirmed root-cause labels.

The reported tuning run achieved average Recall@5 of 1.00, adjusted precision of 0.65, and cross-language hit rate of 1.00 at depth five. Its precision denominator is $\min(5, \text{eligible cases})$, because one query has only two candidates. These figures come from the sample experiment discussed in this submission; they are not independent production benchmarks. The same four queries influenced parameter selection, so a fresh holdout is needed. All query descriptions were English; retrieving German, French and Italian targets does not establish quality for queries in those languages.

I would ask dispatchers or technicians to judge a larger set with graded relevance, negative examples, rare types and every query language. I would compare BM25-only, semantic-only and hybrid retrieval using recall, nDCG, precision and abstention quality. I would rethink fusion if it repeatedly performs worse than semantic-only search, if irrelevant cases dominate, or if first useful output misses p95 three seconds. A later controlled pilot should measure first-visit repair rate, repeat visits and unnecessary parts loaded, with enough observations to separate improvement from noise.

5. What I cut and what comes next

I cut the frontend, generation and follow-ups, reviewed cause labels, calibrated probabilities, reranking, AWS deployment, ingestion jobs and production load testing. The sample is too small to establish calibration or representative performance. The next engineering step is a multilingual reranker plus insufficient-evidence behaviour, tested on fresh queries. The next data step is a reviewed cause dataset. I would not add numerical diagnostic confidence solely to make the prototype look complete.

AI use

I shared my notes and design decisions with AI and used it to help organize, format and improve the wording of this document. I also used AI for brainstorming and draft code and test suggestions. I reviewed and modified the implementation and ran the retrieval experiments myself. AI suggested permanent generated root-cause labels; I kept the original outcomes because unreviewed labels could silently introduce errors. AI also suggested a larger candidate pool, but the sample experiment favoured depth five over ten, so I kept the smaller default.